LETTER

Formal and Empirical Analysis of an Authorization Decision Inconsistency in SAGA

TODO AUTHOR[†], ***??***

SUMMARY Authentication and token secrecy do not ensure receiver-side authorization. SAGA's order-dependent matcher inconsistency yields specification-deny but implementation-allow. An authorization diagnostic exposes a proof-coverage gap. Controlled consumer validation reaches token issuance and storage; under the fixed symbolic divergence, ProVerif reports `ChatAccept` reachable while the non-injective correspondence requiring prior `TokenIssue` holds.

key words: *authorization, access control, formal verification, AI agents, ProVerif*

1. Introduction

LLM-based agents increasingly use external tools and services [1], [2]. Authentication establishes identity; receiver-side authorization determines whether the identified agent may communicate [3]. SAGA [4] combines authenticated identities, contact policies, and access-control tokens. Its existing authentication and token-secrecy results do not establish that token issuance follows the receiver's intended policy.

An authorization coverage diagnostic exposes this proof-coverage gap. It motivates code inspection and controlled matcher tests, which reveal an order-dependent decision inconsistency. We examine its consumer propagation and consequences in a fixed symbolic scenario.

Our contributions are (1) an authorization proof-coverage gap in SAGA's reference model; (2) an order-dependent production matcher inconsistency and controlled consumer propagation to token issuance and storage; and (3) formal consequence analysis showing symbolic `ChatAccept` reachability with a true prior-`TokenIssue` correspondence under a fixed specification-deny/implementation-allow divergence.

(1) Related work.

Prior work studies authorization languages and policy semantics [5], [6] and policy verification and change-impact analysis [7]. SPKI [3] and Macaroons [8] provide credential- and capability-based authorization abstractions. ProVerif and Tamarin support symbolic protocol verification [9], [10]. We complement SAGA's protocol guarantees by examining a concrete policy-evaluation error and its consumer consequences under SAGA's model assumptions.

2. Authorization Decision Inconsistency

(1) Workflow and policy semantics.

SAGA's Provider handles contact requests under the receiver's ordered policy, which maps identity patterns to integer budgets. The Provider and receiver independently call the production matcher: the Provider gates one-time key (OTK) release, and the receiver rechecks authorization before token issuance.

SAGA, Section IV-D.1 [4], specifies selection of the most-specific matching rule; `aid_specificity()` scores the patterns. A positive budget permits the request; a negative budget denies it. We consider fixed policies and valid identities, excluding equal-specificity conflicts and zero budgets.

(2) Root cause and observation.

The tested production matcher is `match()` in `saga/common/contact_policy.py`. The artifact binds the archived matcher observation to pinned upstream source provenance.[†] Its `best_pattern` starts at `None`, whose specificity is -1. A more-specific matching rule updates the budget but never `best_pattern`. Later matches still compare against -1, allowing a less-specific wildcard to overwrite the budget. In the recorded case, `aid_specificity()` scores `alice@example.com:*` as 70 and `*` as 0; their budgets are -1 and 10, respectively.

Table 1 Specification and matcher budgets for two rule orders.

Rule order	Spec. budget	Matcher budget
Specific deny, then wildcard allow	-1	10
Wildcard allow, then specific deny	-1	-1

An independent finite specification evaluator selects the specific denial for both orders in Table 1. Specific deny followed by wildcard allow yields specification budget -1 but matcher budget 10; reversing the order restores denial. This establishes an order-dependent implementation-level authorization bypass in this case.

[†]TODO: affiliation and postal address.

DOI: 10.1587/transfun.E0.A.1

[†]Reproduction artifact: `github.com/g1020268262-a11y/SAGA-Authorization-Analysis-Artifact`.

Copyright © 200x The Institute of Electronics, Information and Communication Engineers

3. Formal Authorization Analysis

(1) Coverage diagnostic.

The archived ProVerif 2.05 diagnostic extends SAGA's reference model. It preserves the original authentication and token-secrecy results but reports an added correspondence requiring prior `PolicyAllow` for `Accept` as false. `Accept` denotes token receipt and verification, distinct from `ChatAccept`. `PolicyAllow` is not connected to policy enforcement; this is an authorization proof-coverage gap. The diagnostic neither executes the Python matcher nor identifies its defect.

(2) Model and assumptions.

The Decision Bridge transfers the checked specification-deny/implementation-allow decision pair into a fixed symbolic scenario. ProVerif assumes this pair; it does not execute Python or the production matcher or discover the defect. The Bridge is neither a general code-to-model translator nor a proof of Python-to-ProVerif refinement. The executions are compared only through authorization decisions and token-lifecycle events.

The initiator has a valid registered identity, follows the prescribed symbolic peer process, and cannot alter the receiver's fixed policy. Reachability is therefore not an arbitrary-adversary claim. The Provider and Certificate Authority (CA) are trusted, cryptographic checks are effective, and private keys are uncompromised. A private symbolic channel abstracts authenticated, confidential transport.

In the symbolic receiver, $\texttt{TokenIssue}(R, I, t)$ follows `ActiveToken` storage of receiver R, initiator I, encrypted token t, and I's public access-control key, and precedes transmission. For private-channel message m, $\texttt{ChatAccept}(R, I, t, m)$ occurs only after checking the fixed identity and an `ActiveToken` entry matching R, I, t, and I's sender public key.

(3) Authorization conditions and results.

The artifact archives the ProVerif 2.05 model and raw output. The run reports `ChatAccept` reachable under the fixed divergence; separately, it verifies the non-injective token-origin correspondence:

$$\mathsf{ChatAccept}(R, I, t, m) \Longrightarrow \mathsf{TokenIssue}(R, I, t)$$

Here R is the receiver and token issuer, and I is the initiator. Prior issuance must match R, I, and encrypted token t. For a static receiver policy, a distinct ideal authorization condition is

$$\mathsf{TokenIssue}(R, I, t) \Longrightarrow \mathsf{Allow}_{\mathrm{spec}}(P_R, I),$$

where P_R is R's contact policy and $\mathsf{Allow}_{\mathrm{spec}}$ denotes permission under its intended semantics, rather than the matcher result. This second formula is a desired condition, neither an executed ProVerif query nor a proved theorem.

For the recorded specific-deny then wildcard-allow case, the correspondence coexists with intended-policy denial. Token-origin consistency therefore does not imply policy-compliant issuance. The `ChatAccept` witness includes issuance; no standalone `TokenIssue` reachability query exists.

4. Consumer-Side Propagation Validation

Archived source hashes bind the matcher, Provider, receiver, and token-related cryptographic code to the pinned upstream baseline. The harness drives the production `/access` route and receiver handler using signed registration material, local certificates, and in-memory transport, retaining production cryptographic checks. Both consumers call the original matcher; wrappers record and return its result unchanged. Fixtures preset neither authorization decisions nor token state.

(1) Cases and controls.

Both consumers use identical policies. D/A places specific deny (-1) before wildcard allow (10); A/A makes both budgets 10, and D/D reverses the D/A order. The letters denote specification and observed implementation decisions, respectively. Each case ran once with fresh state and one OTK. A/A reaches token sending and storage; D/D returns HTTP 403 without OTK release or state change and never reaches the receiver, so receiver rejection remains untested.

Table 2 Production consumer results for the divergent D/A case.

Evidence point	Divergent case result
Specification	Deny (-1)
Provider matcher	Allow (10)
Provider `/access`	HTTP 200; OTK released
Receiver matcher	Allow (10)
Receiver	OTK consumed
Endpoint	Token issued and stored

Table 2 confirms token issuance and storage despite specification denial: one token is generated, sent, and stored in `active_tokens`. No application message is sent: a boundary stub intercepts `receive_conversation` before the original handler executes.

5. Scope and Conclusion

The recorded SAGA inconsistency propagates to token issuance and storage in controlled executions. Under the corresponding fixed symbolic divergence, `ChatAccept` is reachable while its prior-`TokenIssue` correspondence holds. Token-origin consistency cannot substitute for receiver-side authorization consistent with the intended policy.

The evidence covers one recorded case and the tested policies. It establishes neither arbitrary-policy correctness nor an arbitrary-attacker or deployed exploit. Only the symbolic model reaches message acceptance.

References

[1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W.X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," Frontiers of Computer Science, vol.18, no.6, p.186345, 2024.

[2] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," Advances in Neural Information Processing Systems 37 (NeurIPS 2024), pp.82895–82920, 2024.

[3] C. Ellison, B. Frantz, B. Lampson, R. Rivest, B. Thomas, and T. Ylonen, "SPKI certificate theory," RFC 2693, RFC Editor, Sept. 1999. Experimental.

[4] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea, "SAGA: A security architecture for governing AI agentic systems," Network and Distributed System Security (NDSS) Symposium, Internet Society, Feb. 2026.

[5] M.Y. Becker and P. Sewell, "Cassandra: Distributed access control policies with tunable expressiveness," 5th IEEE International Workshop on Policies for Distributed Systems and Networks (POLICY), 2004.

[6] M.Y. Becker, C. Fournet, and A.D. Gordon, "Design and semantics of a decentralized authorization language," 20th IEEE Computer Security Foundations Symposium (CSF), pp.3–15, IEEE, 2007.

[7] K. Fisler, S. Krishnamurthi, L.A. Meyerovich, and M.C. Tschantz, "Verification and change-impact analysis of access-control policies," 27th International Conference on Software Engineering (ICSE), pp.196–205, 2005.

[8] A. Birgisson, J.G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," Network and Distributed System Security Symposium (NDSS), Internet Society, 2014.

[9] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," Foundations and Trends in Privacy and Security, vol.1, no.1–2, pp.1–135, 2016.

[10] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN prover for the symbolic analysis of security protocols," Computer Aided Verification (CAV), Lecture Notes in Computer Science, vol.8044, pp.696–701, Springer, 2013.